Phishing Threat Intelligence Scraper

Web Scraping Project

Module: IT360 information assurance and security

Academic Year: 2025–2026

Project Group

Name	Major
Azza Jomni	FIN-IT
Tassnim Msallem	FIN-IT
Ahmed Rjeb	FIN-IT
Aziz Mdaini	Fin-IT
Taycir Sahnouni	MRK-IT

Generated with LaTeX

Contents

1. High-Level Design

1.1 Architectural Philosophy

The Phishing Threat Intelligence Scraper is designed around a **layered pipeline architecture**: a linear sequence of discrete, loosely coupled components in which each layer consumes the output of its predecessor, applies a single well-defined transformation, and forwards a richer artifact to the next layer.

This architectural pattern was selected deliberately for three reasons. First, *separation of concerns* ensures that the URL collection logic, scraping logic, parsing logic, scoring logic, storage logic, and presentation logic are each contained in a single module, making the codebase readable and testable by all group members independently. Second, *replaceability* means that any individual component can be upgraded without touching the rest of the system — for example, the static `requests` engine could be swapped for a browser-based driver if the threat scope were ever extended to JavaScript-rendered pages. Third, the *linear data flow* from raw URL to visualised risk score is directly traceable, which satisfies both academic reproducibility requirements and the audit trail obligations common to real-world security tooling.

1.2 Layered Pipeline Architecture

The diagram below illustrates the full end-to-end pipeline. Data enters the system as a list of candidate URLs obtained from public phishing databases and exits as a classified, timestamped record rendered in an interactive web dashboard.

At each stage, the artifact passing through the pipeline gains semantic richness: a bare URL string becomes an HTTP response object; the HTML bytes become a structured list of threat indicators; those indicators become a numeric score and categorical risk label; the score becomes a database record; and the record becomes a human-readable visual report.

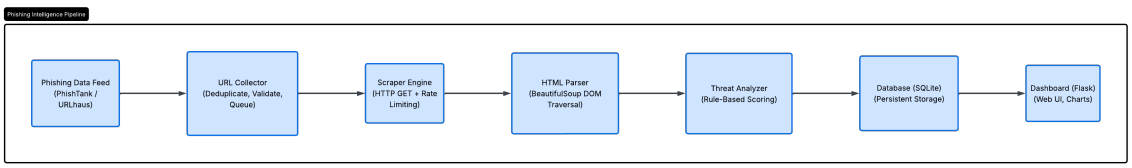

Figure 1: End-to-end layered pipeline architecture of the phishing threat intelligence system.

1.3 Components

The following subsections describe each of the six pipeline components in detail, including their inputs, internal logic, outputs, and design rationale.

1.3.1 URL Feed

Purpose. The URL Feed is the entry point of the pipeline. Its sole responsibility is to obtain a fresh list of candidate suspicious URLs and deliver them in a normalised format to the Scraper Engine.

Data Sources. The system integrates two primary public feeds. *PhishTank* provides a community-verified CSV export of confirmed phishing URLs, updated several times daily. *URLhaus*, maintained by the Abuse.ch threat research team, offers a continuously updated CSV of malicious URLs. Both datasets are freely accessible for non-commercial and academic research.

Processing Logic:

1. Download the latest CSV export via an HTTP GET request to the feed endpoint.
2. Parse each row, extracting the URL column while discarding metadata fields such as submission timestamps and reporter identity.
3. Apply basic sanitisation: strip whitespace, enforce `http://` or `https://` scheme, and reject malformed entries.
4. Deduplicate the resulting list to avoid redundant scraping of the same target.
5. Yield each validated URL as a string to the Scraper Engine.

Output. A duplicated Python list of URL strings, e.g., `["http://phish-example.com/login", ...]`.

Attribute	Value
Input	PhishTank CSV feed / URLhaus CSV file
Output	Deduplicated list of validated URL strings
Key operations	HTTP download, CSV parsing, deduplication, format validation

1.3.2 Scraper Engine

Purpose. The Scraper Engine performs the HTTP negotiation between the analysis system and each target web server. It is the only component with outbound network access.

Core Mechanism. For each URL received from the feed, the engine dispatches an HTTP GET request. A descriptive `User-Agent` header is included in every request to identify the scraper as a research tool, in accordance with responsible disclosure norms. The engine captures the full HTTP response, including the status code, response headers, and the raw HTML body.

Resilience and Rate Limiting. The engine wraps each request in a `try/except` block to catch `ConnectionError`, `Timeout`, and `TooManyRedirects` exceptions. A configurable timeout (default: 10 seconds) prevents the engine from hanging on unresponsive hosts. A randomised `time.sleep()` delay of between 1 and 3 seconds is introduced between consecutive requests to comply with ethical scraping practices and avoid triggering anti-bot protections.

Attribute	Value
Input	Validated URL list from the URL Feed
Output	2-tuple of `(status_code: int, html_content: str)`, or `None` on failure
Responsible	Student 2
Key operations	`requests.get()`, timeout handling, random sleep, error logging

1.3.3 Component 3 — HTML Parser

Purpose. The HTML Parser transforms the raw HTML string received from the Scraper Engine into a structured indicator list by performing DOM traversal.

DOM Traversal Strategy. The parser builds an in-memory tree representation of the HTML document, enabling navigation by tag name, attribute, CSS class, or CSS selector. It operates entirely on the static source HTML without executing JavaScript. For each page, the parser queries the DOM for the following threat-relevant element categories:

- **Iframes:** all `<iframe>` tags, inspecting their `style` attribute for `display:none` or

`visibility:hidden`.

- **Forms:** all `<form>` tags, extracting the `action` attribute and comparing its domain against the host domain of the page URL.
- **Input fields:** all `<input>` tags with `type="password"`.
- **External scripts:** all `<script>` tags with a `src` attribute pointing to a domain not matching the page host.
- **Text content:** the full visible text of the page, searched for urgency keywords using case-insensitive string matching.
- **Anchor links:** all `<a>` tags, checking whether their `href` values use the insecure `http://` scheme.
- **Page title:** the `<title>` tag text, compared against a dictionary of known brand names to detect impersonation.

Output. A Python list of indicator dictionaries, e.g., `[{"indicator": "hidden_iframe", "details": "..."}, ...]`, forwarded to the Threat Analyzer.

Attribute	Value
Input	Raw HTML string from Scraper Engine
Output	List of indicator dicts: `{indicator, score, detail}`
Responsible	Student 3
Key operations	`BeautifulSoup(html, 'html.parser')`, `.find_all()`, attribute inspection

1.3.4 Component 4 — Threat Analyzer

Purpose. The Threat Analyzer is the intelligence core of the system. It applies a deterministic, rule-based scoring engine to the indicator list produced by the parser and emits a final risk classification for each URL.

Scoring Model. Each indicator is mapped to a numeric score weight calibrated to reflect its empirical association with confirmed phishing activity. Score weights are additive: the total score for a URL is the sum of the weights of all detected indicators. Table 1 presents

the complete ruleset.

Table 1: Threat scoring rules and indicator weights.

Indicator	Detection Rule	Score	Severity
Hidden iframe	`<iframe>` with `display:none` or `visibility:hidden`	+3	High
External login form	`<form action>` points to a different domain	+3	High
Password field	`<input type="password">` present on page	+2	Medium
Suspicious script	`<script src>` loads from external or unknown domain	+2	Medium
Urgency keywords	Words such as *verify now*, *suspended*, *click immediately*	+1	Low
No HTTPS links	All internal anchors use `http://`	+1	Low
Mismatched title	Page title references a brand absent from the URL host	+2	Medium

Risk Classification Thresholds:

- **LOW (0–3):** no confirmed high-severity indicators; page may be suspicious but lacks strong phishing signals.
- **MEDIUM (4–6):** one or more medium-severity indicators detected; the page warrants further investigation.
- **HIGH (7+):** multiple high-severity indicators detected; the page strongly resembles a confirmed phishing attack.

Design Rationale. A rule-based approach was chosen over a machine-learning classifier because it requires no labelled training dataset, every decision is fully explainable and auditable, and the ruleset can be extended by adding new rows to the scoring table without retraining. The trade-off is reduced generalisability to novel phishing techniques not yet encoded in the rules.

Attribute	Value
Input	Indicator list from HTML Parser
Output	Result object: `{url, score, risk_level, indicators, timestamp}`
Responsible	Student 3
Key operations	Score accumulation, threshold mapping, result object construction

1.3.5 Component 5 — SQLite Database

Purpose. The Database layer provides durable, queryable storage of all analysis results. It decouples the scraping and analysis pipeline from the presentation layer, enabling the dashboard to operate independently and to support historical queries across multiple scraping runs.

Schema Overview. The relational schema is normalised to third normal form (3NF) and comprises three tables: `urls`, `results`, and `scores`. The design separates URL metadata from indicator-level results and summary scores, reflecting a one-to-many relationship between a URL and its detected indicators. The complete schema is documented in Section 1.4.

Attribute	Value
Input	Scored result object from Threat Analyzer
Output	Persisted records; query responses to Dashboard
Responsible	Student 4
Key operations	`CREATE TABLE`, `INSERT INTO`, `SELECT` with `JOIN`, parameterised queries

1.3.6 Component 6 — Flask Dashboard

Purpose. The Flask Dashboard is the user-facing output of the pipeline. It reads from the SQLite database and renders analysis results as a filterable, sortable HTML table with summary statistics, enabling analysts to review, search, and export findings.

Route Structure. The Flask application exposes the following endpoints:

- `/` (index): renders a summary view with aggregate statistics — total URLs scraped, risk level distribution, most recently analysed entries.
- `/results`: renders the full results table, supporting query parameters for filtering by risk level and date range.
- `/results/<id>`: renders the detail view for a single URL, listing all detected indicators and their individual scores.
- `/export`: returns a downloadable CSV of the full results table.

Templating. Jinja2 templates are used to separate presentation logic from Python route handlers. Bootstrap CSS is included via a CDN link to provide a responsive layout without requiring a front-end build step.

Attribute	Value
Input	Query results from Database
Output	HTML web interface served on `localhost:5000`
Responsible	Student 5
Key operations	Flask routes, Jinja2 templates, SQLite queries, result rendering

1.4 Database Schema

The database is composed of three related tables. The `urls` table is the root entity; `results` and `scores` each hold a foreign key `url_id` referencing `urls.id`. This normalised structure avoids data duplication and allows indicator-level analysis to be queried independently of summary scores.

1.4.1 Table: `urls`

Stores one row per scraping event, capturing the identity and HTTP metadata of a target URL at the moment of analysis.

Table 2: Schema for the `urls` table.

Column	Type	Constraint	Description
`id`	`INTEGER`	PRIMARY KEY	Auto-incrementing surrogate key
`url`	`TEXT`	NOT NULL	Full URL of the analysed page, including scheme
`scraped_at`	`DATETIME`	NOT NULL	UTC timestamp of the scrape event, set on INSERT
`status_code`	`INTEGER`	—	HTTP response status code (e.g., 200, 403, 404)

1.4.2 Table: `results`

Stores one row per detected indicator per URL. A single URL may produce multiple rows — one for each phishing signal identified during DOM analysis.

Table 3: Schema for the `results` table.

Column	Type	Constraint	Description
`id`	`INTEGER`	PRIMARY KEY	Auto-incrementing surrogate key
`url_id`	`INTEGER`	FOREIGN KEY → urls.id	Links each indicator to its parent URL record
`indicator`	`TEXT`	NOT NULL	Canonical name of the detected indicator
`score`	`INTEGER`	NOT NULL	Integer score weight contributed by this indicator

1.4.3 Table: `scores`

Stores one row per URL, holding the aggregated threat assessment. Populated after all indicators for a URL have been inserted into `results`, acting as a materialised summary.

Table 4: Schema for the `scores` table.

Column	Type	Constraint	Description
`url_id`	`INTEGER`	PK / FK → urls.id	One-to-one relationship with the `urls` record
`total_score`	`INTEGER`	NOT NULL	Arithmetic sum of all `results.score` values for this URL
`risk_level`	`TEXT`	NOT NULL	Categorical label: `LOW`, `MEDIUM`, or `HIGH`

1.4.4 Data Lifecycle

When the pipeline processes a URL, the following INSERT sequence executes within a single database transaction to ensure atomicity:

```
import sqlite3, datetime

def store_result(url, status_code, indicators, total_score,
    risk_level):
    conn   = sqlite3.connect("phishing_intel.db")
    cursor = conn.cursor()

    # 1. Insert URL metadata
    cursor.execute(
        "INSERT INTO urls (url, scraped_at, status_code) VALUES (?,
            ?, ?)",
        (url, datetime.datetime.utcnow(), status_code)
    )
    url_id = cursor.lastrowid

    # 2. Insert each detected indicator
    for item in indicators:
        cursor.execute(
            "INSERT INTO results (url_id, indicator, score) VALUES
                (?, ?, ?)",
            (url_id, item["indicator"], item["score"])
        )

    # 3. Insert aggregated score
    cursor.execute(
        "INSERT INTO scores (url_id, total_score, risk_level) VALUES
            (?, ?, ?)",
```

```
        (url_id, total_score, risk_level)
    )

    conn.commit()
    conn.close()
```

Listing 1: Database insert sequence for a single URL analysis result.

The use of parameterised queries throughout prevents SQL injection — particularly important given that the data being inserted originates from untrusted external web pages.

2. Tools & Technology Stack

2.1 Overview and Selection Criteria

The selection of every tool in this stack was governed by four explicit criteria:

1. **Technical fit:** the tool must fulfil the component's functional requirements without architectural workarounds.
2. **Minimal dependency footprint:** each additional library adds installation complexity, potential version conflicts, and attack surface. The stack was kept as lean as possible.
3. **Accessibility and team competency:** all group members must be able to read, write, and debug code using the chosen tools within the project timeframe.
4. **Community support and longevity:** tools with active maintenance, comprehensive documentation, and large user communities reduce the risk of encountering unresolvable issues.

Table 5: Complete tools and technology stack with versions and roles.

Tool	Version	Role	Pipeline Component
Python	3.11+	Core programming language	All components
Requests	2.31+	HTTP client for URL fetching	Scraper Engine
BeautifulSoup4	4.12+	HTML DOM parsing	HTML Parser
lxml	4.9+	High-performance parser backend	HTML Parser
SQLite3	Built-in	Relational database	Database
Flask	3.0+	Web framework for dashboard	Dashboard
Jinja2	3.1+	HTML templating engine	Dashboard
PhishTank API	Public	Verified phishing URL feed	URL Feed
URLhaus CSV	Public	Malicious URL feed	URL Feed
GitHub	—	Version control, submission	Development

2.2 Python 3.11+

Python was the unambiguous language choice for this project. Its dominance in both data engineering and cybersecurity tooling means that every component of the pipeline — HTTP networking, HTML parsing, relational database access, and web framework development — is served by mature, well-documented libraries.

Python 3.11 was targeted specifically because its improved error messages reduce debugging time for less experienced developers, and it delivers meaningful performance improvements over Python 3.9 and 3.10. The alternative of using JavaScript (Node.js with Puppeteer) was considered and rejected: the target class of phishing pages are predominantly static HTML, making full JavaScript execution overhead unnecessary, and the team's strongest collective competency is in Python.

2.3 Requests

A central architectural decision in any web scraping project is whether to use a static HTTP client or a headless browser. The correct choice is determined entirely by the nature of the target content.

Phishing pages are, by design, structurally simple. Analysis of the PhishTank and URLhaus datasets confirms that the overwhelming majority of phishing pages deliver their malicious HTML structures — hidden iframes, credential-harvesting forms, urgency text — directly in the initial HTTP response, without JavaScript-driven DOM mutation. `Requests` is therefore the appropriate tool.

`Requests` is a synchronous HTTP client library that provides a clean interface for constructing requests and inspecting responses. It does not launch a browser process, consumes negligible CPU and memory, supports session management for persistent cookies and headers, and provides a well-specified exception model for robust error handling.

Selenium and Playwright were evaluated and rejected: both require downloading and managing a full browser binary, add significant overhead per page load, and introduce complexity that yields no technical benefit for this use case. Scrapy, a complete scraping framework, was also considered but rejected as architectural over-engineering for a single-pipeline project of this scope.

2.4 BeautifulSoup4

BeautifulSoup4 is a Python library that parses HTML documents into a navigable in-memory tree, allowing code to locate and extract specific elements by tag, attribute, or CSS selector. It was selected for three practical reasons.

First, it handles malformed HTML gracefully — a necessary quality given that phishing pages are often hastily constructed and may contain missing closing tags, improperly nested elements, or invalid attribute values. Second, it exposes flexible query interfaces: `find()` and `find_all()` for tag-based searches, `select()` for CSS selectors, and direct attribute access via dictionary notation. Third, it supports a pluggable parser backend, allowing the faster `lxml` engine to be used in place of the default `html.parser` with no changes to the analysis code.

2.5 SQLite3

SQLite is a file-based relational database engine included in Python's standard library, requiring no installation and no separate server process. The database is stored as a single file on the local filesystem, making it trivial to deploy alongside the scraper and straightforward to share between group members.

For the access patterns of this project — sequential batch inserts during scraping runs followed by analytical read queries from the dashboard — SQLite's performance is entirely sufficient. It supports full SQL including joins and aggregations, and the portability of the database file means a populated dataset can be included directly in the project submission.

2.6 Flask

Flask is a lightweight Python web framework built on the Werkzeug WSGI toolkit. It provides URL routing via decorators, access to the SQLite database within route handlers, HTML template rendering through Jinja2, and the ability to return file downloads — covering everything required for the dashboard layer with a minimal API surface.

Flask was preferred over Django because the project requires only a read-heavy single-database dashboard. Django's built-in ORM, migration framework, and admin interface would add unnecessary complexity. Flask's explicit, un-opinionated design allows every layer of the request–response cycle to remain visible and understandable, which is valuable for a team learning the framework.

2.7 Phishing Data Sources

The feed layer relies on two external, publicly maintained phishing databases, vetted for free academic access, machine-readable format, regular update cadence, and community trust.

- **PhishTank** provides community-submitted and verified phishing URLs. Its bulk CSV download delivers up to several hundred thousand entries per export, each row containing the URL, a verification timestamp, and a confirmed-phishing flag. The system filters for confirmed entries only.
- **URLhaus** is maintained by Abuse.ch and catalogues URLs distributing malware and command-and-control endpoints. Its CSV export is updated in near-real-time and requires no API key, making it highly accessible for research environments.

3. Development Phases

The project is divided into eight sequential phases, each producing a concrete, testable deliverable and mapping to a named team member.

#	Phase	Key Tasks	Deliverable
1	Project Setup	Create GitHub repo, install dependencies, configure folder structure, share requirements.txt	Working local environment
2	URL Feed Integration	Connect to PhishTank, download URLhaus CSV, parse feeds, deduplicate, validate URL format	url_collector.py
3	Scraper Development	Implement GET requests with timeout, add rate-limiting sleep, handle 4xx/5xx/SSL errors, log failures	scraper.py
4	Threat Detection	Define indicator rules, implement BeautifulSoup queries, build scoring engine, map score to risk level	parser.py + analyzer.py
5	Database Integration	Create SQLite schema, implement INSERT/SELECT functions, connect analyzer output to DB writes	database.py
6	Dashboard Development	Create Flask routes, write Jinja2 templates, display URL table, add risk-level badges and statistics	app.py
7	Integration & Testing	Connect all modules, run end-to-end tests with real and mock URLs, fix bugs, verify DB records	Functional integrated system
8	Documentation	Finalise report, write README, comment code, upload deliverables, share repository with teacher	Report + GitHub repository

3.1 GitHub Repository Structure

```
phishing-scraper/
 README.md                <- Project overview and setup instructions
 requirements.txt         <- Python dependency list
 main.py                  <- Entry point: runs the full pipeline
 url_collector.py         <- Phase 2: URL feed integration
 scraper.py               <- Phase 3: HTTP scraping engine
 parser.py                <- Phase 4: HTML DOM parsing
 analyzer.py              <- Phase 4: Threat scoring logic
 database.py              <- Phase 5: SQLite operations
 app.py                   <- Phase 6: Flask dashboard
 templates/
    dashboard.html       <- Jinja2 HTML template
 data/
    sample_urls.csv      <- Static test URL dataset
 docs/
     report.pdf           <- Final compiled report
```